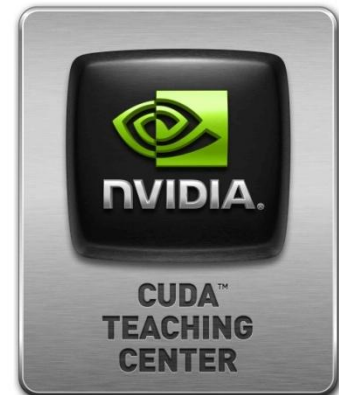


Programación masivamente paralela en GPUS (pmpGPU)

M. Freire, E. Dufrechou,
P. Ezzatti y M. Pedemonte



Clase 12

Abstracciones y Bibliotecas en CUDA Moderno

Motivación

- **Nuevas abstracciones simplifican la programación paralela.**
- **Las bibliotecas permiten reutilizar implementaciones optimizadas.**
- **Muchas rutinas pueden expresarse mediante primitivas paralelas reutilizables.**
- **Existe un compromiso entre generalidad, productividad y performance.**
- **El conocimiento del hardware continúa siendo fundamental.**

Agenda

- **Cooperative Groups**
- **Tensor Cores**
- **Building Blocks**
- **Bibliotecas CUDA**
 - **Ejemplos**
- **CUDA Graphs**

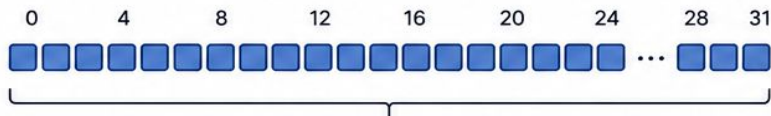
Cooperative groups

Cooperative groups

Comparación del modelo de sincronización

CUDA clásico

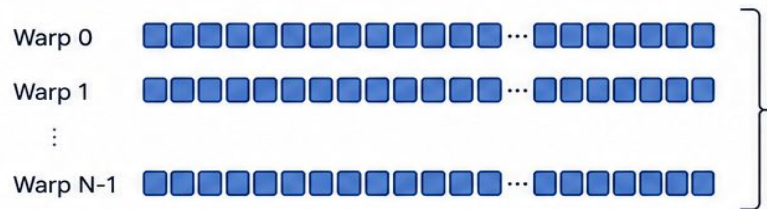
Warp (32 threads)



`__syncwarp()`

Sincronización dentro del warp completo

Block (múltiples warps)



`__syncthreads()`

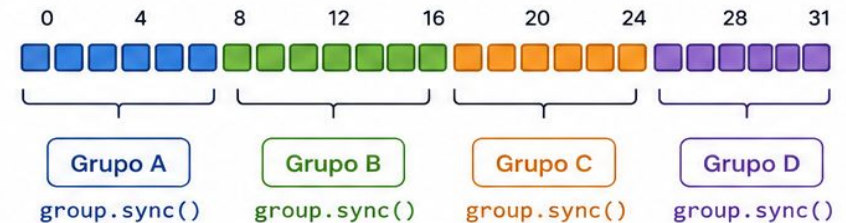
Sincronización de todo el bloque



- Alcance de sincronización rígido (warp o bloque)
- Difícil expresar cooperación más fina
- Manejo manual para patrones complejos

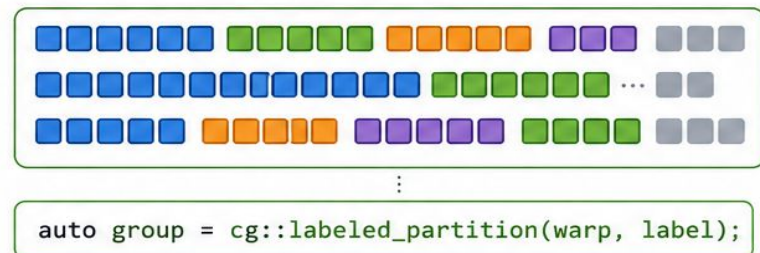
Cooperative Groups

Grupos flexibles



- Cada grupo tiene su propia sincronización
- Los grupos pueden formarse dinámicamente

Particiones dentro de un bloque



- Grupos de threads flexibles y expresivos
- Sincronización y cooperación adaptadas al problema
- Menos manejo manual, más claridad en el código

Cooperative groups

- **Creación y sincronización de grupos cooperativos**
 - **thread_block**
 - **coalesced_group**
 - **grid_group**
- **Particionado estático de threads: tiled_partition<N>**
- **Particionado dinámico según datos**
 - **labeled_partition**
 - **binary_partition**
- **Operaciones colectivas dentro del grupo: sync, reduce, shfl**

Cooperative groups - Ejemplo

Volvamos al ejemplo del práctico 3

- Reducciones en paralelo en el warp
 - Positivo: max
 - Negativo: suma
- Es complejo hacerlo con shuffles para patrones irregulares
- ¿Cómo queda con cooperative groups?

Cooperative groups - Ejemplo

Cooperación dependiente de los datos

1. Warp original (32 lanes)

lane:	0	1	2	3	4	5	6	7	...	24	25	26	27	28	29	30	31
valor:	+3	-2	+7	-1	+9	-4	-8	+5	...	-6	+2	+1	-3	+4	-7	+6	-5
	positivo	negativo															

2. Dos objetivos paralelos



Positivos: máximo

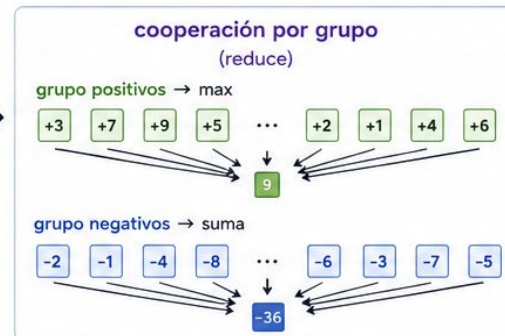
$$\max(+3, +7, +9, +5, +2, +1, +4, +6) = 9$$



Negativos: suma

$$(-2) + (-1) + (-4) + (-8) + (-6) + (-3) + (-7) + (-5) = -36$$

3. Con Cooperative Groups (automático y expresivo)



resultados
(por grupo)



máximo = 9



suma = -36

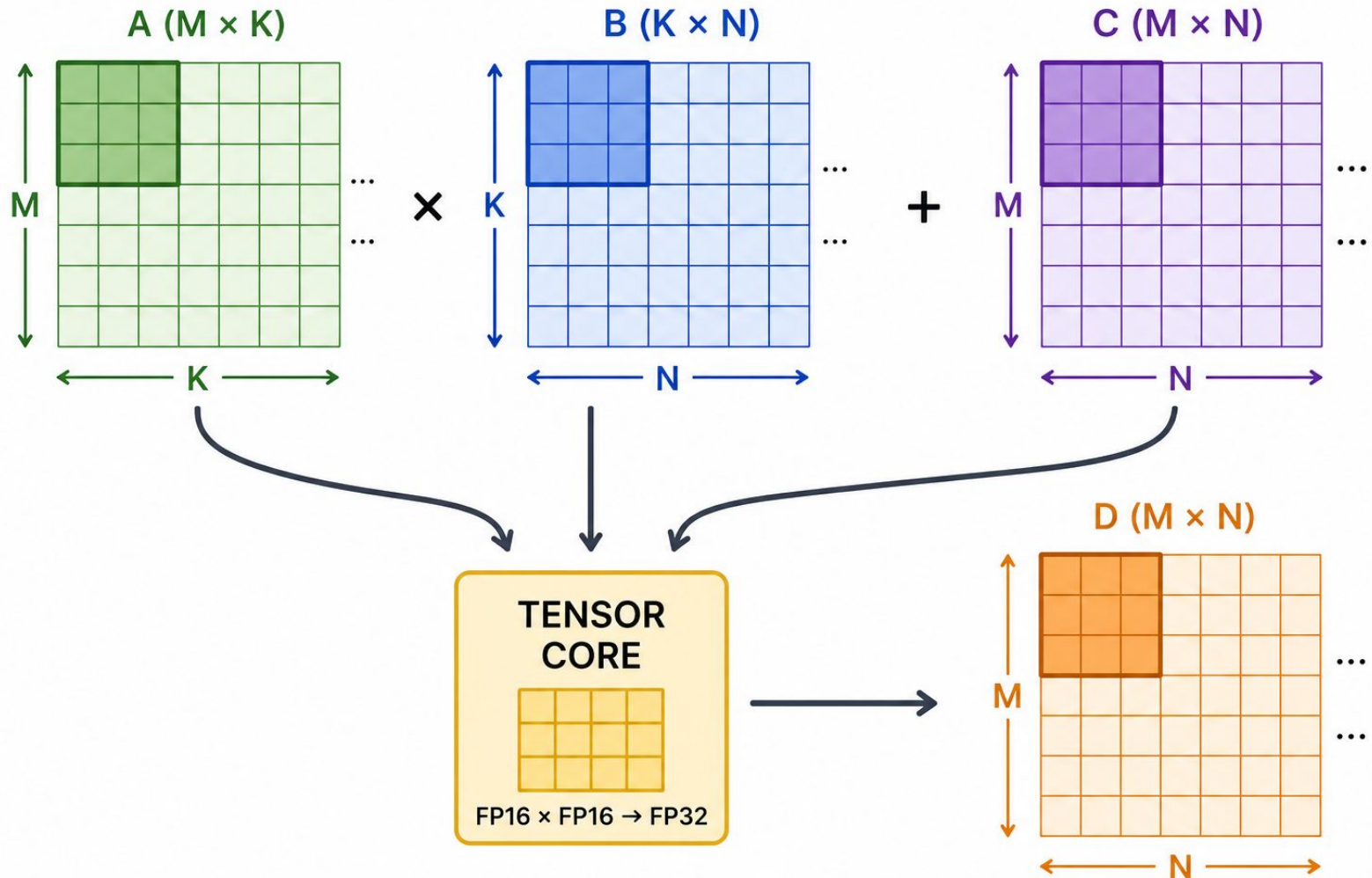
```
// 1) Particionar el warp según el signo (etiqueta)
auto block = cg::this_thread_block();
auto tile32 = cg::tiled_partition<32>(block);
auto group = cg::labeled_partition(tile32, x > 0);
// true -> positivos, false -> negativos
```

```
// 2) Cooperar dentro de cada grupo
int resultado = 0;
if (x > 0)
    resultado = cg::reduce(group, x, cg::maximum<int>());
else
    resultado = cg::reduce(group, x, cg::plus<int>());
```

Tensor cores

Tensor cores

Problema



Tensor Cores

- La multiplicación de bloques densos es muy repetida
 - Actualmente redes neuronales
- Se agrega un chip para hacer esta operación
- En general no se requiere tanta precisión
 - En Turing FP16 x FP16 $\rightarrow$ 32
- Se puede programar directamente (wmma) o usarlo a través de bibliotecas (cublas)

Tensor Cores - WMMA

- Interfaz a nivel de warp
 - **wmma::fragment**: define entrada y salida
 - **wmma::load**: copiar a la entrada
 - **wmma::mma_sync**: multiplicar
 - **wmma::fill_fragment**: dar valor
 - **wmma::store_matrix_sync**: guardar resultado en memoria

Tensor Cores - WMMA

```
wmma::fragment<wmma::matrix_a, 16, 16, 16, half,  
                wmma::row_major> a_frag;  
wmma::fragment<wmma::matrix_b, 16, 16, 16, half,  
                wmma::row_major> b_frag;  
wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;  
  
// Inicializar acumulador en 0.0  
wmma::fill_fragment(c_frag, 0.0f);  
  
// Cargar matrices A y B (tiles 16x16)  
wmma::load_matrix_sync(a_frag, A, 16);  
wmma::load_matrix_sync(b_frag, B, 16);  
  
// C = A * B + C  
wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);  
  
// Guardar resultado en matriz C (tile 16x16)  
wmma::store_matrix_sync(C, c_frag, 16, wmma::mem_row_major);
```

Building blocks

Building blocks

- En general se piensan como funciones sobre vectores o conjuntos
- Algunos ejemplos de estos “building blocks” son:
 - Copias
 - Transformaciones
 - Reducciones
 - Scan
 - Ordenar
 - Merge
 - Permutaciones
 - Search

Copias

- **Contiguas**
 - **Normal**
 - **Host-Device**
- **Reordenamientos**
 - **Gather**
 - **Scatter**

Gather

Fuente
(vector origen)

10	20	30	40	50	60	70	80	90	100	110	120	130	140	150	160
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Map
(índices)

2	0	4	1	9	1
0	1	2	3	4	5

Resultado
(vector destino)

30	10	50	20	100	20
0	1	2	3	4	5

Gather

Fuente
(vector origen)

10	20	30	40	50	60
----	----	----	----	----	----

Map
(índices destino)

2	0	4	1	9	6
---	---	---	---	---	---

Resultado
(vector destino)

20	40	10	0	30	0	60	0	0	50
----	----	----	---	----	---	----	---	---	----

Transformaciones

- **Aplicar funciones en paralelo**
 - **Foreach**
 - **Generalmente in-place**
 - **Funciones sencillas**
 - **Transform**
 - **Generalmente out-of-place**
 - **Funciones de múltiples parámetros**

Transformaciones

FOREACH ($x^2 + 1$)

Entrada (A):	-3	-2	-1	0	1	2		→	10	5	2	1	2	5	10
	0	1	2	3	4	6			0	1	2	3	4	5	6

Paso	Índice i	A[i] = x	Operación	B[i] = $x^2 + 1$
1	0	-3	$(-3)^2 + 1$	10
2	1	-2	$(-2)^2 + 1$	5
...	...	...	...	...
7	6	3	$(3)^2 + 1$	10

TRANSFORM (suma elemento a elemento)

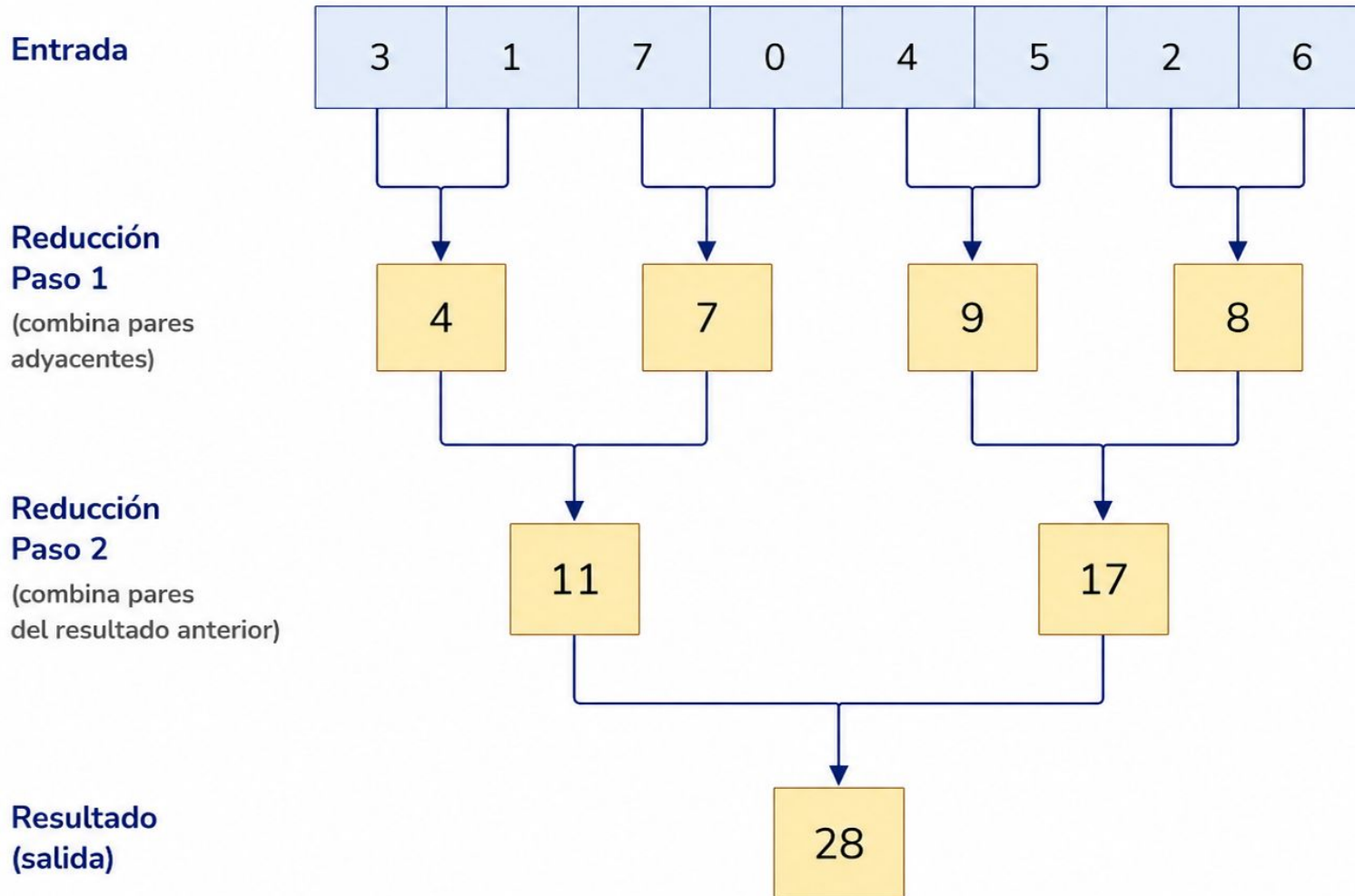
Entrada A:	1	2	3	4	5	6	7		Salida C: $c[i] = a[i] + b[i]$							
	0	1	2	3	4	5	6	→	11	22	33	44	55	66	77	
Entrada B:	10	20	30	40	50	60	70			0	1	2	3	4	5	6
	0	1	2	3	4	5	6									

Paso	Índice i	A[i]	B[i]	Operación	C[i] = A[i] + B[i]
1	0	1	10	1 + 10	11
2	1	2	20	2 + 20	22
...	...	...	...	...	...
7	6	7	70	7 + 70	77

Reducciones

- **Acumula los elementos en un único valor usando una operación binaria**
 - **Suma**
 - **Max**
 - **Min**
 - **Otras...**

Reducciones



Scan

- **Acumula los elementos de un vector manteniendo los resultados parciales**
 - **Exclusivo: no incluye valor actual**
 - **Inclusivo: incluye el valor actual**
- **Se debe definir operación (suma, mult, etc.) y valor inicial (típicamente 0 o 1)**

Scan

Entrada

(vector de entrada)

3	1	7	0	4	5	2	6
---	---	---	---	---	---	---	---

Paso 1

(stride = 1)

Cada elemento suma el anterior

3	4	8	7	4	9	7	8
---	---	---	---	---	---	---	---

Cada elemento i ($i > 0$) suma el elemento $i-1$

Paso 2

(stride = 2)

Cada elemento suma el de dos posiciones anteriores

3	4	11	11	12	16	11	17
---	---	----	----	----	----	----	----

Cada elemento i ($i \geq 2$) suma el elemento $i-2$ (resultado anterior)

Paso 3

(stride = 4)

Cada elemento suma el de cuatro posiciones anteriores

3	4	11	11	15	20	22	28
---	---	----	----	----	----	----	----

Cada elemento i ($i \geq 4$) suma el elemento $i-4$ (resultado anterior)

Salida (inclusiva)

(suma prefija inclusiva)

3	4	11	11	15	20	22	28
---	---	----	----	----	----	----	----

Salida (exclusiva)

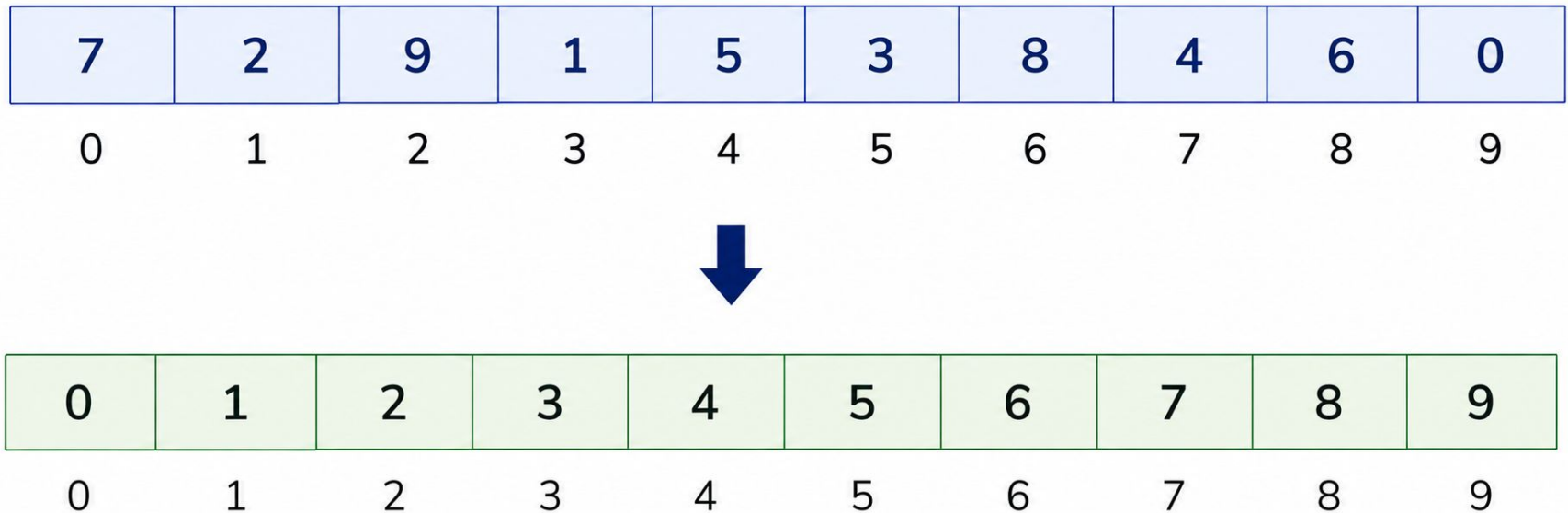
(suma prefija exclusiva)

0	3	4	11	11	15	20	22	28
---	---	---	----	----	----	----	----	----

Ordenar

- **Ordena un vector**
 - **Puede mantener ordenes parciales (stable) o no**
 - **Puede ser segmentado**

Ordenar



Permutaciones

- **Reordenar elementos de un vector**
 - **Aleatorias: Aplicaciones de algoritmos estocásticos (xej: Métodos Montecarlo)**
 - **Particiones: Clasificación binaria (xej: condición es par)**

Permutaciones

PERMUTACIÓN ALEATORIA

Entrada:

8	3	5	1	7	2	9	4	6
0	1	2	3	4	5	6	7	8



Salida:

5	8	2	9	1	6	3	7	4
0	1	2	3	4	5	6	7	8

PARTICIÓN PARES E IMPARES

Entrada:

7	2	9	4	1	6	3	8	5
0	1	2	3	4	5	6	7	8



Salida:

2	4	6	8	7	9	1	3	5
0	1	2	3	4	5	6	7	8

PARES

IMPARES

Merge

- **Unir dos vectores ordenados**

Merge

1	3	5	7	9
0	1	2	3	4

2	4	6	8	10
0	1	2	3	4



1	2	3	4	5	6	7	8	9	10
0	1	2	3	4	5	6	7	8	9

Filtrado

- Copiar solamente parte del vector
 - Típicamente se usa para compactar

Filtrado

COPY_IF (condición: > 0)

Vector de entrada (A):

-3	5	0	2	-1	7	0	4
0	1	2	3	4	5	6	7

Condición: $A[i] > 0$

Vector de salida (B):

5	2	7	4
0	1	2	3

Paso	i	A[i]	A[i] > 0	¿Cumple?	B (parcial)								
1	0	-3	-3 > 0	no	<table><tr><td>-</td><td>-</td><td>-</td><td>-</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>	-	-	-	-	0	1	2	3
-	-	-	-										
0	1	2	3										
2	1	5	5 > 0	sí	<table><tr><td>5</td><td>-</td><td>-</td><td>-</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>	5	-	-	-	0	1	2	3
5	-	-	-										
0	1	2	3										
...	...	...	...	...	...								
8	7	4	4 > 0	sí	<table><tr><td>5</td><td>2</td><td>7</td><td>4</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td></tr></table>	5	2	7	4	0	1	2	3
5	2	7	4										
0	1	2	3										

Búsqueda

- **Encontrar el índice de cierto elemento**
 - **Binaria**
 - **Iterativa**
 - **Mismatch**

Búsqueda

BÚSQUEDAS: BINARIA E ITERATIVA

Vector ordenado (A):

2	8	15	23	38
0	1	2	3	4

Valor a buscar (x):

23

1. BÚSQUEDA BINARIA

Paso	izq	der	medio	A[medio]	Comparación	Acción										
1	0	4	2	15	$15 < 23$	ir derecha										
<table><tr><td>2</td><td>8</td><td>15</td><td>23</td><td>38</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>							2	8	15	23	38	0	1	2	3	4
2	8	15	23	38												
0	1	2	3	4												
2	3	4	3	23	$23 = 23$	¡encontrado!										
<table><tr><td>2</td><td>8</td><td>15</td><td>23</td><td>38</td></tr><tr><td>0</td><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>							2	8	15	23	38	0	1	2	3	4
2	8	15	23	38												
0	1	2	3	4												

Resultado: posición 3

2. BÚSQUEDA ITERATIVA (LINEAL)

Paso	i	A[i]	Comparación	Acción
1	0	2	$2 \neq 23$	siguiente
2	1	8	$8 \neq 23$	siguiente
3	2	15	$15 \neq 23$	siguiente
4	3	23	$23 = 23$	¡encontrado!

Resultado: posición 3

Búsqueda

MISMATCH

Dos vectores:

Compara elemento a elemento
y devuelve la posición donde
difieren.

A:

4	2	7	9	1	5	8	3
0	1	2	3	4	5	6	7

B:

4	2	7	9	0	5	8	3
0	1	2	3	4	5	6	7

Resultado:

posición 4

A[4] = 1, B[4] = 0

Paso	Índice i	A[i]	B[i]	Comparación	Resultado parcial
1	0	4	4	4 = 4	iguales, continuar
2	1	2	2	2 = 2	iguales, continuar
...	...	...	...	...	...
5	4	1	0	1 ≠ 0	diferentes

Bibliotecas

Bibliotecas

- **CUDA incluye bibliotecas muy optimizadas**
- **Muchas primitivas comunes ya tienen implementaciones eficientes**
 - **Típicamente muy buenas para casos standard**
- **Existen bibliotecas de distinto nivel de abstracción**
- **El programador elige entre productividad y control**

Bibliotecas

Tipos de bibliotecas CUDA

BUILDING BLOCK LIBRARIES

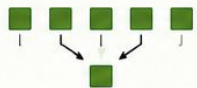
Primitivas paralelas de propósito general



Thrust

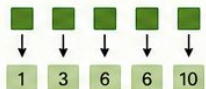


CUB



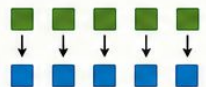
reduce

Agrega elementos en paralelo



scan (prefix sum)

Suma acumulativa en paralelo



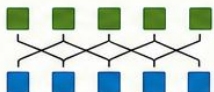
transform

Aplica una operación a cada elemento



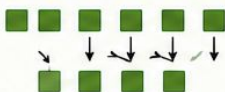
copy_if (filtrado/compactación)

Selecciona elementos que cumplen una condición



gather / scatter

Accesos indirectos y reordenamiento de datos



sort, radix_sort

Ordenamiento paralelo eficiente

BIBLIOTECAS NUMÉRICAS

Rutinas especializadas de alto rendimiento



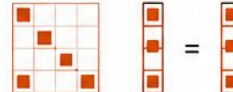
cuBLAS



Álgebra lineal densa
(GEMM, vectores, etc.)



cuSPARSE



Operaciones con matrices
dispersas
(SpMV, factorizaciones, etc.)



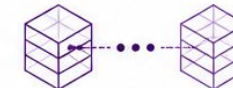
cuFFT



Transformada rápida
de Fourier
(1D, 2D, 3D)



cuTENSOR



Operaciones con tensores
(contracciones, permutaciones)



Otras bibliotecas

NCCL, cuRAND, cuSOLVER,
cuDNN, etc.

Bibliotecas - Thrust

- **Extensión de STL para GPUs**
- **Manejo automático de memoria y ejecución**
- **Fácil de usar para programadores sin mucha práctica (o que la pierdan!)**
- **Tiene muchas primitivas paralelas**
- **Interoperabilidad con el resto de CUDA**

Bibliotecas - Thrust

- Manejo de memoria automático (con contenedores) o manual (usando wrappers de punteros)
- La liberación de memoria es automática (pero es más eficiente hacerla manual!!!)
- Los wrappers mantienen información (tipo, dispositivo, etc.)
- Se puede pasar de contenedor a puntero usando `.data()` y casteo a `raw_pointer`

Bibliotecas - CUB

- Surge de Thrust por necesidades de mantenimiento, portabilidad y escalabilidad
- Un poco más bajo nivel que thrust
- Específica para CUDA C++
- Maneja memoria igual que CUDA (cudaMalloc, cudaFree, punteros) y corre por el programador
- Implementación (más) eficiente de los principales algoritmos

Bibliotecas - CUB

- Se dividen en tres granularidades
 - Warp
 - Block
 - Device
- Además de los BB mencionados presenta otras cosas específicas de problemas tanto a nivel de bloque (discontinuity, adjacent difference) como device (select, runLength encoding, SpMV)
- Soporta problemas batch (xej: segmented sort)

Ejemplo reducciones

Manual

```
extern __shared__ long int intermedio[];
int idx = blockIdx.x * blockDim.x + threadIdx.x;
if (idx < sz)
    intermedio[threadIdx.x] = input[idx];
else
    intermedio[threadIdx.x] = 0;

__syncthreads();
int i = blockDim.x / 2;
while (i != 0) {
    if (threadIdx.x < i)
        intermedio[threadIdx.x] =
            intermedio[threadIdx.x] +
            intermedio[threadIdx.x + i];
    __syncthreads();
    i = i / 2;
} // endwhile

__syncthreads();
if (threadIdx.x == 0 && idx < sz)
    output[blockIdx.x] = intermedio[0];
```

Thrust

```
#include <thrust/device_ptr.h>
#include <thrust/reduce.h>

long int thrust_reduce_sum
const long int* d_input,
long int n

const long int init = 0

thrust::device_ptr<const long int> first(d_input);
thrust::device_ptr<const long int> last = first + n;

return thrust::reduce(first, last, init,
    thrust::plus<long int>());
```

CUB

```
#include <cub/cub.cuh>

long int cub_reduce_sum
const long int* d_input,
long int n
{
    long int* d_output = nullptr;
    size_t temp_storage_bytes = 0;

    cub::DeviceReduce::Sum(nullptr, temp_storage_bytes,
        d_input, d_output, n);

    void* d_temp_storage;
    cudaMalloc(&d_temp_storage, temp_storage_bytes);

    cub::DeviceReduce::Sum(d_temp_storage, temp_storage_bytes,
        d_input, d_output, n);

    long int h_output;
    cudaMemcpy(&h_output, d_output, sizeof(long int),
        cudaMemcpyDeviceToHost);

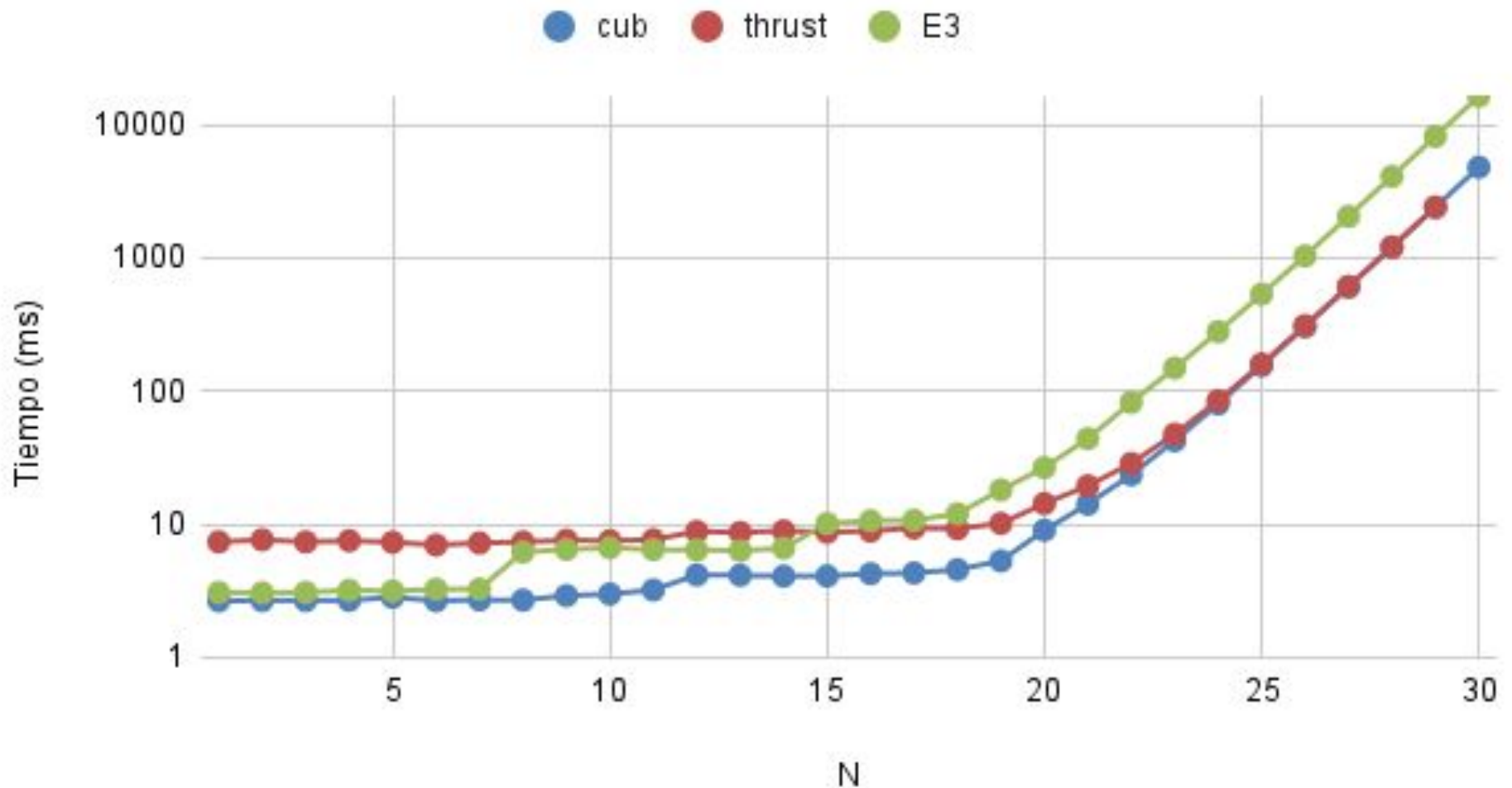
    cudaFree(d_temp_storage);
    cudaFree(d_output);
    return h_output;
}
```

Ejemplo reducciones

- Se ejecutó cada algoritmo 500 veces en una tarjeta Nvidia RTX 3090 Ti
- Se evaluó el tiempo de ejecución ignorando la primera ejecución (overhead de las bibliotecas)
- Se probaron distintos tamaños medios y grandes ($1024 * 2^N$)
- Thrust no ejecuta con el valor de $N=30$

Ejemplo reducciones

Reduce ejecutado en RTX 3090 Ti (500 ejec)



Ejemplo reducción

- ¿Qué pasa si quiero usar otro operador?
- Hay varios implementados como maximo, mínimo, etc.
- ¿Y si no?

```
struct Mul7Plus {  
    __host__ __device__  
    long long operator()(long long a,  
                          long long b) const {  
        return a * 7 + b;  
    }  
};
```

Ejemplo compresión de matrices

- Las bibliotecas no solo sirven para paralelizar primitivas, también se puede programar rutinas complejas combinándolas
- Tomemos como ejemplo el cargado de matrices dispersas
 - Generalmente se guardan como una lista de los valores con sus índices de fila o columna (COO)
 - Esta idea se puede mejorar comprimiendo un índice (CSR)

Ejemplo compresión de matrices

1. MATRIZ DISPERSA (Densa)

Ejemplo de matriz 5 x 6

	0	1	2	3	4	5
0	0	0	3	0	0	0
1	0	9	0	0	4	0
2	0	0	0	7	0	0
3	8	0	0	0	0	5
4	0	0	0	0	0	6

Valores distintos de cero

2. FORMATO COO (Coordenadas)

Cada elemento no nulo se almacena como (fila, columna, valor)

fila (row)	col (col)	valor (val)
0	2	3
1	1	9
1	4	4
2	3	7
3	0	8
3	5	5
4	5	6

nnz = 7 (cantidad de no nulos)

Tamaño de almacenamiento: $3 * \text{nnz}$

3. FORMATO CSR (Compressed Sparse Row)

Valores y columnas de los no nulos, más el arreglo de punteros a filas (**row_ptr**)

val (valores no nulos)

3	9	4	7	8	5	6
---	---	---	---	---	---	---

col_ind (índices de columna)

2	1	4	3	0	5	5
---	---	---	---	---	---	---

row_ptr (puntero al inicio de cada fila)

0	1	3	4	6	7
---	---	---	---	---	---

0 1 2 3 4 5
↑ = nnz (7)

nnz = 7 (cantidad de no nulos)

Tamaño de almacenamiento: $\text{nnz} + (\text{n_filas} + 1)$

Ejemplo compresión de matrices

COO (entrada)

rowIdx (nnz)

2	0	2	1	0	2
0	1	2	3	4	5

colIdx (nnz)

3	1	0	2	4	5
0	1	2	3	4	5

values (nnz)

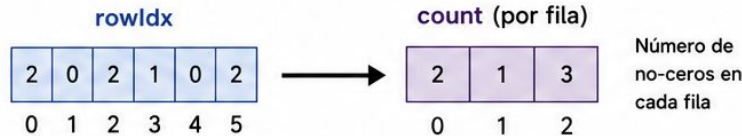
10	20	30	40	50	60
0	1	2	3	4	5

Ejemplo de matriz 3x6
con nnz = 6

	0	1	2	3	4	5
0		20			50	
1			40			
2	30			10		60

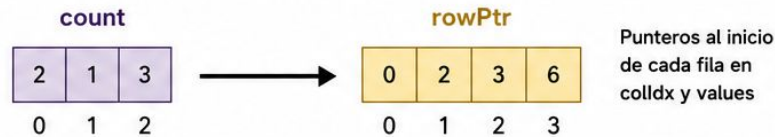
1 Contar elementos por fila (Building Block #1)

Histograma de rowIdx



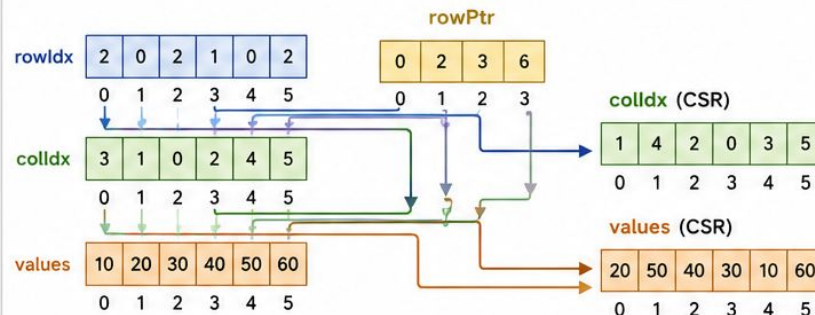
2 Suma prefija (Building Block #2)

Exclusive prefix sum sobre count



3 Reordenar columnas y valores (Building Block #3)

Usando rowPtr como offsets (scatter)



CSR (salida)

rowPtr (nRows+1)

0	2	3	6
0	1	2	3

colIdx (nnz)

1	4	2	0	3	5
0	1	2	3	4	5

values (nnz)

20	50	40	30	10	60
0	1	2	3	4	5

Misma matriz en CSR

	0	1	2	3	4	5
0		20			50	
1			40			
2	30			10		60

Ejemplo compresión de matrices

```
// Sort by row
thrust::device_vector<int> sorted_indices(d_row.size());
thrust::sequence(sorted_indices.begin(), sorted_indices.end());
thrust::sort_by_key(d_row.begin(), d_row.end(), sorted_indices.begin());

// Sort columns and data according to the sorted row indices
thrust::device_vector<int> d_col_sorted(d_col.size());
thrust::device_vector<int> d_data_sorted(d_data.size());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_col.begin(), d_col_sorted.begin());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_data.begin(),
d_data_sorted.begin());

// Count the number of elements per row
thrust::device_vector<int> row_counts(d_row.size());
thrust::constant_iterator<int> const_iter(1);
auto new_end = thrust::reduce_by_key(d_row.begin(), d_row.end(), const_iter,
thrust::make_discard_iterator(), row_counts.begin());
int num_unique_rows = new_end.second - row_counts.begin();
```

Ejemplo 2 – Compresión de matrices

● ● ● Vector de 1 a N

```
// Sort by row
thrust::device_vector<int> sorted_indices(d_row.size());
thrust::sequence(sorted_indices.begin(), sorted_indices.end());
thrust::sort_by_key(d_row.begin(), d_row.end(), sorted_indices.begin());

// Sort columns and data according to the sorted row indices
thrust::device_vector<int> d_col_sorted(d_col.size());
thrust::device_vector<int> d_data_sorted(d_data.size());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_col.begin(), d_col_sorted.begin());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_data.begin(),
d_data_sorted.begin());

// Count the number of elements per row
thrust::device_vector<int> row_counts(d_row.size());
thrust::constant_iterator<int> const_iter(1);
auto new_end = thrust::reduce_by_key(d_row.begin(), d_row.end(), const_iter,
thrust::make_discard_iterator(), row_counts.begin());
int num_unique_rows = new_end.second - row_counts.begin();
```

Ejemplo 2 – Compresión de matrices

● ● ● Vector de 1 a N

Ordeno el vector usando la fila como clave
Para cada entrada el vector sorted dice su
posición ordenada

```
// Sort by row
thrust::device_vector<int> sorted_indices(d_row.size());
thrust::sequence(sorted_indices.begin(), sorted_indices.end());
thrust::sort_by_key(d_row.begin(), d_row.end(), sorted_indices.begin());

// Sort columns and data according to the sorted row indices
thrust::device_vector<int> d_col_sorted(d_col.size());
thrust::device_vector<int> d_data_sorted(d_data.size());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_col.begin(), d_col_sorted.begin());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_data.begin(),
d_data_sorted.begin());

// Count the number of elements per row
thrust::device_vector<int> row_counts(d_row.size());
thrust::constant_iterator<int> const_iter(1);
auto new_end = thrust::reduce_by_key(d_row.begin(), d_row.end(), const_iter,
thrust::make_discard_iterator(), row_counts.begin());
int num_unique_rows = new_end.second - row_counts.begin();
```

Ejemplo 2 – Compresión de matrices

● ● ● Vector de 1 a N

Ordeno el vector usando la fila como clave
Para cada entrada el vector sorted dice su
posición ordenada

```
// Sort by row
thrust::device_vector<int> sorted_indices(d_row.size());
thrust::sequence(sorted_indices.begin(), sorted_indices.end());
thrust::sort_by_key(d_row.begin(), d_row.end(), sorted_indices.begin());

// Sort columns and data according to the sorted row indices
thrust::device_vector<int> d_col_sorted(d_col.size());
thrust::device_vector<int> d_data_sorted(d_data.size());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_col.begin(), d_col_sorted.begin());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_data.begin(),
d_data_sorted.begin());

// Count the number of elements per row
thrust::device_vector<int> row_counts(d_row.size());
thrust::constant_iterator<int> const_iter(1);
auto new_end = thrust::reduce_by_key(d_row.begin(), d_row.end(), const_iter,
thrust::make_discard_iterator(), row_counts.begin());
int num_unique_rows = new_end.second - row_counts.begin();
```

Uso sorted para ordenar los otros dos vectores

Ejemplo 2 – Compresión de matrices

● ● ● Vector de 1 a N

Ordeno el vector usando la fila como clave
Para cada entrada el vector sorted dice su
posición ordenada

```
// Sort by row
thrust::device_vector<int> sorted_indices(d_row.size());
thrust::sequence(sorted_indices.begin(), sorted_indices.end());
thrust::sort_by_key(d_row.begin(), d_row.end(), sorted_indices.begin());

// Sort columns and data according to the sorted row indices
thrust::device_vector<int> d_col_sorted(d_col.size());
thrust::device_vector<int> d_data_sorted(d_data.size());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_col.begin(), d_col_sorted.begin());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_data.begin(),
d_data_sorted.begin());

// Count the number of elements per row
thrust::device_vector<int> row_counts(d_row.size());
thrust::constant_iterator<int> const_iter(1);
auto new_end = thrust::reduce_by_key(d_row.begin(), d_row.end(), const_iter,
thrust::make_discard_iterator(), row_counts.begin());
int num_unique_rows = new_end.second - row_counts.begin();
```

Uso sorted para ordenar los otros dos vectores

Vector de N 1s

Ejemplo 2 – Compresión de matrices

● ● ● Vector de 1 a N

Ordeno el vector usando la fila como clave
Para cada entrada el vector sorted dice su
posición ordenada

```
// Sort by row
thrust::device_vector<int> sorted_indices(d_row.size());
thrust::sequence(sorted_indices.begin(), sorted_indices.end());
thrust::sort_by_key(d_row.begin(), d_row.end(), sorted_indices.begin());

// Sort columns and data according to the sorted row indices
thrust::device_vector<int> d_col_sorted(d_col.size());
thrust::device_vector<int> d_data_sorted(d_data.size());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_col.begin(), d_col_sorted.begin());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_data.begin(),
d_data_sorted.begin());
```

Uso sorted para ordenar los otros dos vectores

```
// Count the number of elements per row
thrust::device_vector<int> row_counts(d_row.size());
thrust::constant_iterator<int> const_iter(1);
auto new_end = thrust::reduce_by_key(d_row.begin(), d_row.end(), const_iter,
thrust::make_discard_iterator(), row_counts.begin());
int num_unique_rows = new_end.second - row_counts.begin();
```

En counts queda el número
de elementos por fila

Vector de N 1s

Ejemplo 2 – Compresión de matrices

● ● ● Vector de 1 a N

Ordeno el vector usando la fila como clave
Para cada entrada el vector sorted dice su
posición ordenada

```
// Sort by row
thrust::device_vector<int> sorted_indices(d_row.size());
thrust::sequence(sorted_indices.begin(), sorted_indices.end());
thrust::sort_by_key(d_row.begin(), d_row.end(), sorted_indices.begin());

// Sort columns and data according to the sorted row indices
thrust::device_vector<int> d_col_sorted(d_col.size());
thrust::device_vector<int> d_data_sorted(d_data.size());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_col.begin(), d_col_sorted.begin());
thrust::gather(sorted_indices.begin(), sorted_indices.end(), d_data.begin(),
d_data_sorted.begin());
```

Uso sorted para ordenar los otros dos vectores

```
// Count the number of elements per row
thrust::device_vector<int> row_counts(d_row.size());
thrust::constant_iterator<int> const_iter(1);
auto new_end = thrust::reduce_by_key(d_row.begin(), d_row.end(), const_iter,
thrust::make_discard_iterator(), row_counts.begin());
int num_unique_rows = new_end.second - row_counts.begin();
```

En counts queda el número
de elementos por fila

Vector de N 1s

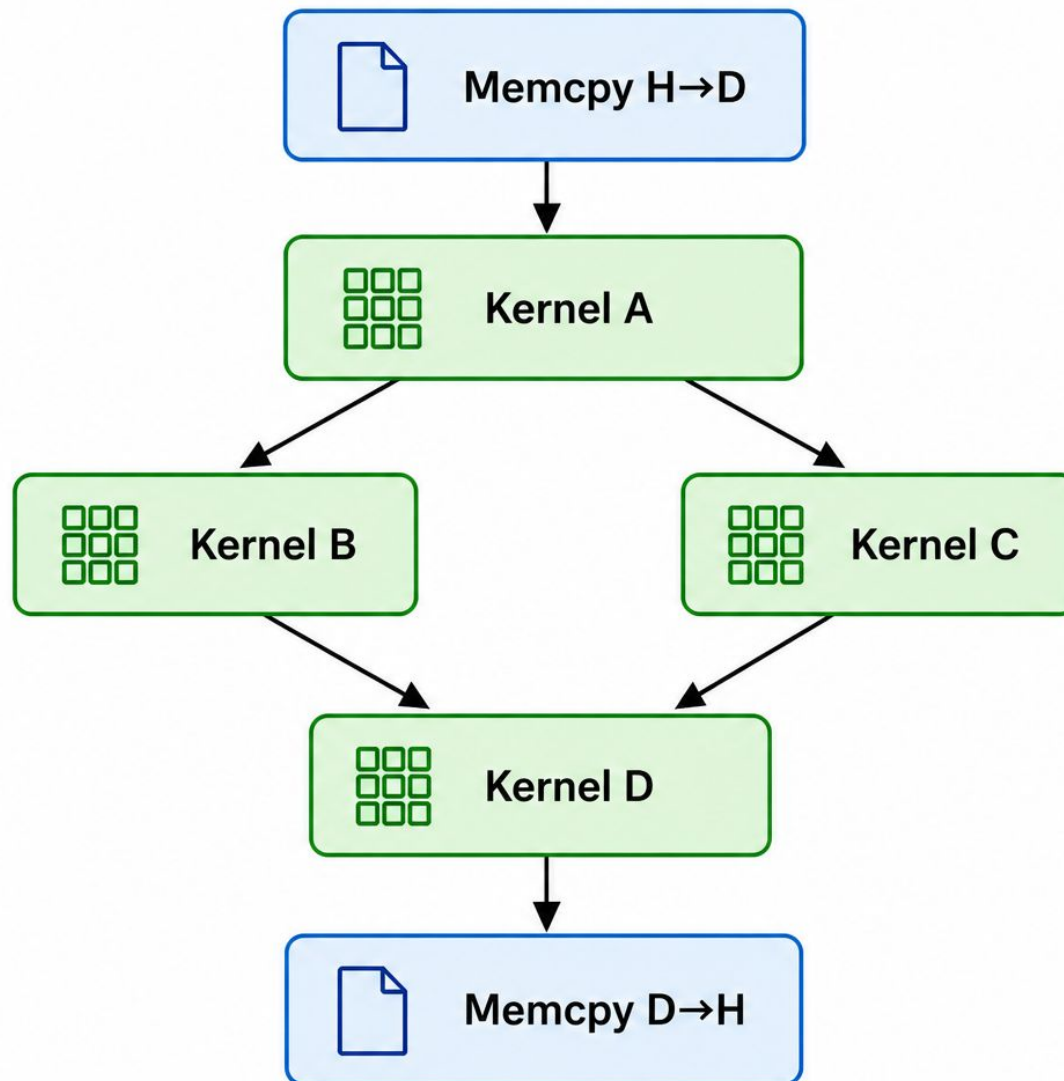
New_end es un puntero al final, restar
me retorna cuántas filas distintas hay

Cuda Graphs

Cuda graphs

- Cuando una aplicación usa muchos kernels cortos el overhead de lanzamiento se vuelve relevante
- En GPUs modernas se puede tirar más de un kernel a la vez usando streams pero a veces hay dependencias
- La idea de los grafos es capturar las dependencias
 - Reduce el overhead de lanzamientos
 - Tiene un costo de generar el grafo la primera vez

Cuda graphs



Cuda graphs

Captura de grafo

```
cudaStream_t stream;
cudaStreamCreate(&stream);

// 1) Inicio de la captura en el stream
cudaStreamBeginCapture(stream;
    cudaStreamCaptureModeGlobal);

// 2) Operaciones que se capturan en el grafo
// Copia Host -> Device
cudaMemcpyAsync(d_input, h_input, bytes,
    cudaMemcpyHostToDevice, stream); // H->D

// Kernels
kernelA<<<gridA, blockA, 0, stream>>>(d_input, d_tmp);
kernelB<<<gridB, blockB, 0, stream>>>(d_tmp, d_output);

// Copia Device -> Host
cudaMemcpyAsync(h_output, d_output, bytes,
    cudaMemcpyDeviceToHost, stream); // D->H

// 3) Fin de la captura: se obtiene el grafo
cudaGraph_t graph;
cudaStreamEndCapture(stream, &graph);

// 4) Instanciar y lanzar el grafo
cudaGraphExec_t graphExec;
cudaGraphInstantiate(&graphExec, graph, NULL, NULL, 0);
cudaGraphLaunch(graphExec, stream);

cudaStreamSynchronize(stream);
```

Definición explícita

```
cudaSraph_t graph;
cudaGraphCreate(&graph, 0);

// 1) Agregar nodos al grafo
cudaGraphAddMemcpyNode(...); // H -> D
cudaGraphAddKernelNode(...); // Kernel A
cudaGraphAddKernelNode(...); // Kernel B
cudaGraphAddMemcpyNode(...); // D -> H
...

// 2) Instanciar y lanzar el grafo
cudaGraphExec_t exec;
cudaGraphInstantiate(&exec, graph, 0);
cudaGraphLaunch(exec, stream);
...
```